

Project Report



Project Title: Gamified App For Student

Author: Shuai Jiang

Supervisor: Joseph Kehoe

Date: 18/04/2026

1. Introduction.....	3
2. Project Achievements.....	5
2.1 Productivity Features.....	5
2.2 Gamification and Progression.....	7
2.3 Nearby Social Features.....	7
2.4 Profile and Avatar Features.....	8
3. Unmet Objectives.....	10
4. Challenges and Resolutions.....	11
4.1 Bluetooth Identity and Nearby Discovery.....	11
4.2 Deployment and Media Persistence.....	11
4.3 Reward Integrity and Anti-Abuse Design.....	12
4.4 Validation on Mobile Devices.....	12
5. Testing and Validation.....	13
5.1 Backend and Functional Validation.....	13
5.2 Device-Level Validation.....	13
5.3 Validation of Reward Integrity.....	13
6. Reflections and Lessons Learned.....	15
6.1 Generation of Features Is Easier Than Validation of Features.....	15
6.2 Platform Reality Matters.....	15
6.3 Full-Stack Projects Reward Clear Separation of Responsibility.....	15
6.4 Scope Control Is a Technical Skill.....	15
7. Future Recommendations.....	16
7.1 Stronger Authentication and Account Verification.....	16
7.2 Expanded Social Confirmation Options.....	16
7.3 Broader Real-Device Bluetooth Testing.....	16
7.4 Richer Social History and Analytics.....	16
7.5 Further Reward-System Tuning.....	16
8. Conclusion.....	17
9. Bibliography.....	18
10. Appendices.....	19
Appendix A: Core Data Structures.....	19
Appendix B: Main Backend Endpoints.....	20

1. Introduction

This report reflects on the complete development of Meet & Merit, a mobile application designed to support student productivity, motivation, and real-world social interaction. The project combines academic planning tools with gamified progression and a nearby social feature intended to encourage students to connect offline rather than spend more time in conventional online messaging systems. It was developed as a full-stack system with an Android frontend in Kotlin and a Django backend deployed to Heroku with PostgreSQL.

The project was motivated by a practical problem. Students often use multiple disconnected tools for tasks, notes, schedules, and communication, yet few systems combine these functions in a way that is specifically designed for student life. In addition, while many applications encourage digital interaction, very few attempt to support structured, in-person campus social engagement. Meet & Merit was intended as a response to both of these gaps. The goal was not simply to build a planner, but to build a student-centred application that rewards productive behaviour and supports meaningful offline social contact.

The Functional Specification, Design Document, and Research Document already describe the planned features, research context, and technical design decisions in detail. This final report therefore focuses less on repeating those documents and more on evaluating what was actually achieved, what remained incomplete, which technical challenges were most significant, how the system was validated, and what lessons emerged from the work.

From a technical perspective, the system combines several areas of software engineering in one project: Android UI development, REST API design, database modelling, cloud deployment, media handling, BLE-based nearby discovery, and XP-driven game logic. The final result is a deployed backend and a working Android application that together demonstrate both feature breadth and iterative technical refinement.

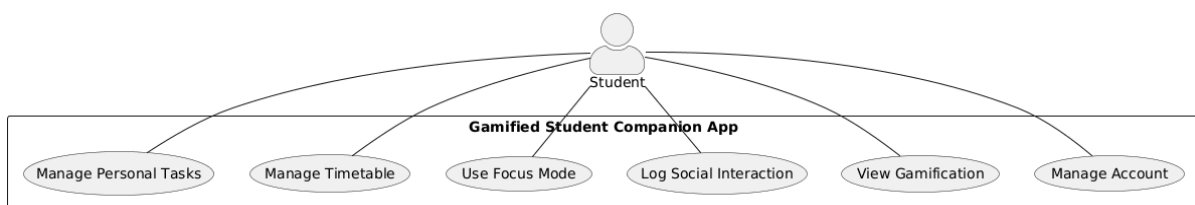


Figure 1. High-level use case diagram for Meet & Merit.

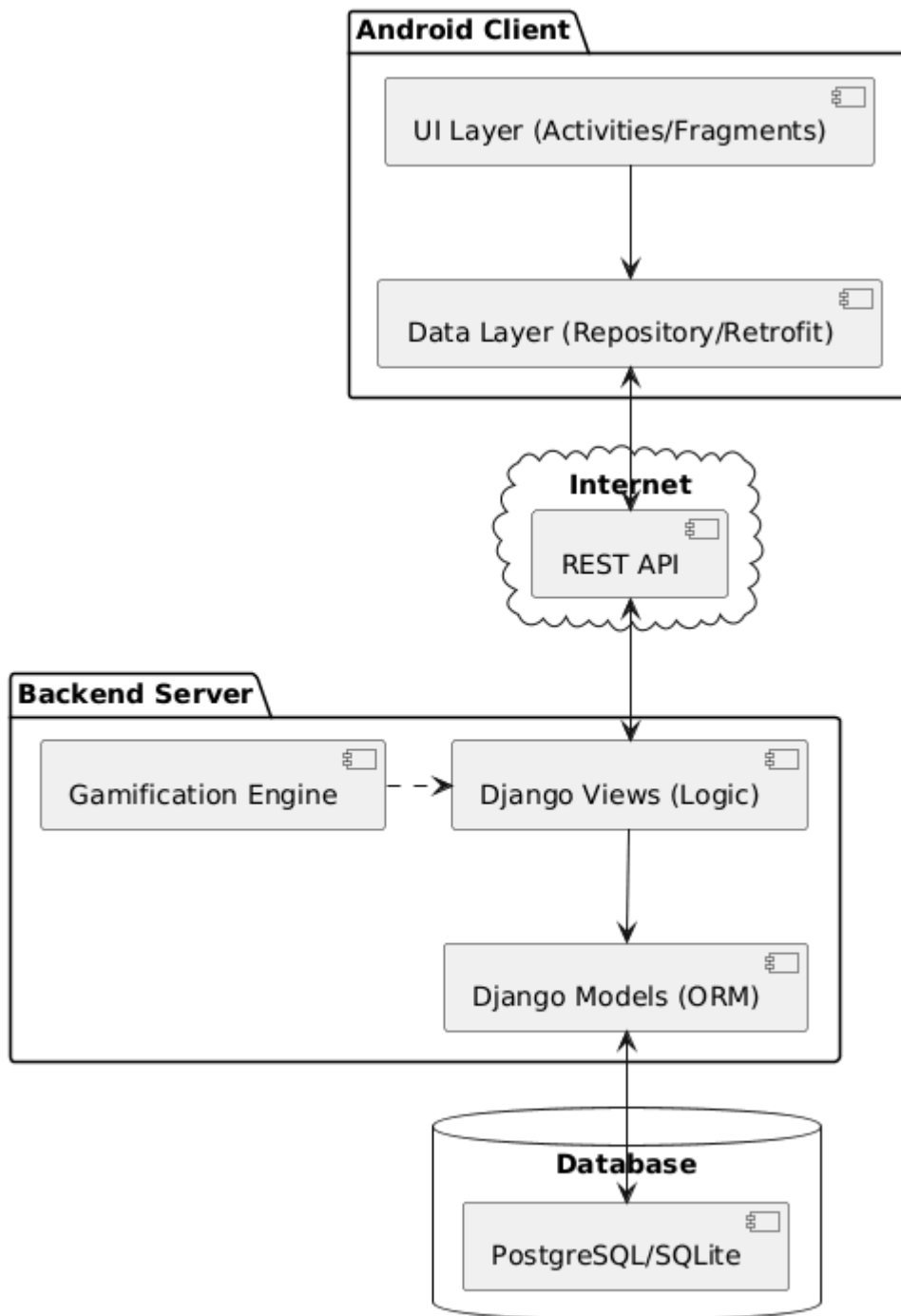


Figure 2. Overall system architecture showing the relationship between frontend, backend, and supporting services.

2. Project Achievements

The project delivered a substantial working prototype rather than an isolated technical demo. Its most important achievements were not only the number of implemented screens or API endpoints, but the fact that the major systems work together as one coherent student application. The final version of Meet & Merit includes a productivity layer, a gamified progression layer, a profile and identity layer, and a nearby social layer. Together these form a much stronger result than a set of disconnected features.

2.1 Productivity Features

The productivity side of the application forms the foundation of the project. Students can create and manage tasks, record focus sessions, organise timetable entries, and maintain notes. These features were designed to feel practical first and gamified second. The app is usable as a student planning tool even without the reward systems, which was an important design goal.

Task management allows students to create, complete, and remove study tasks, while focus mode rewards sustained attention rather than only end results. The timetable and notebook features make the system more student-specific than a generic to-do list because they support class scheduling and academic note-taking directly within the same environment.

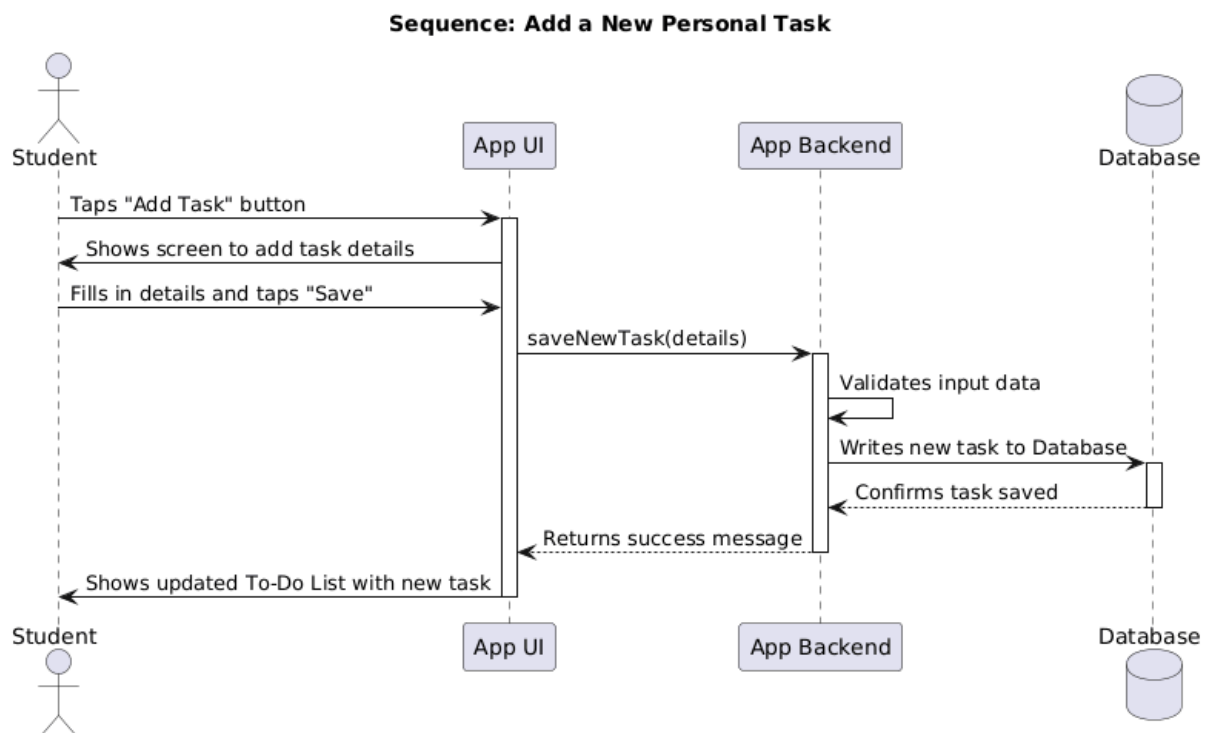


Figure 3. Task creation interface used to add new academic tasks.

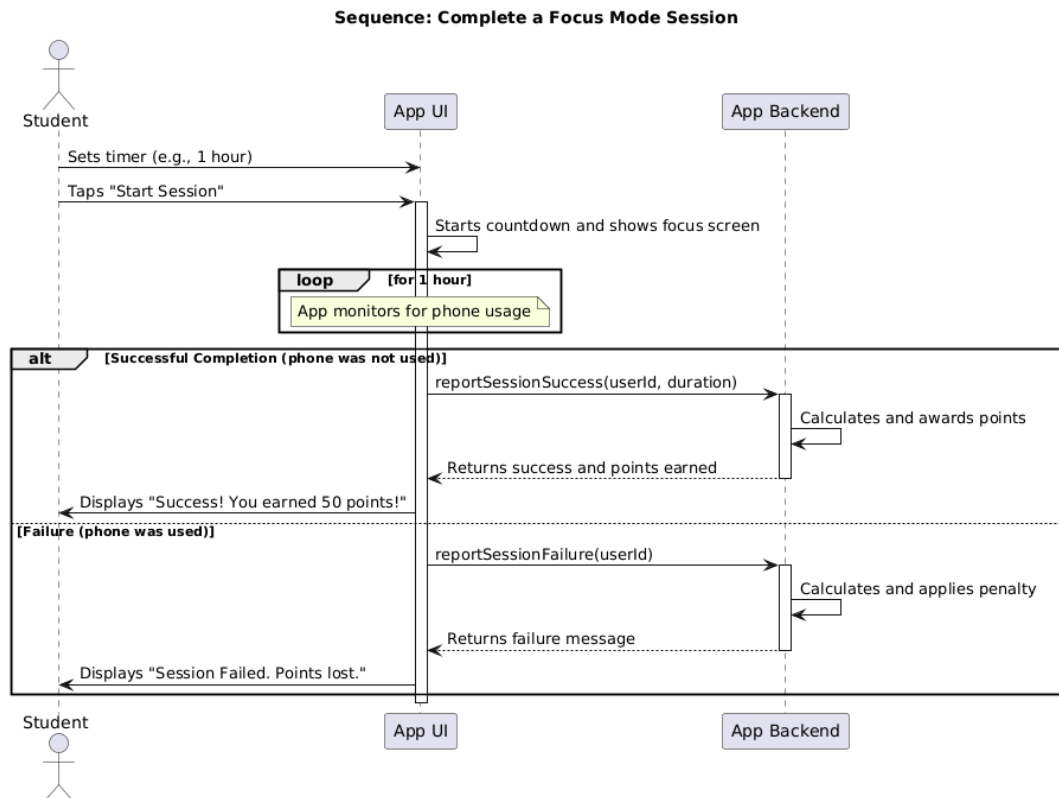


Figure 4. Focus mode screen used to log concentrated study time and award XP.

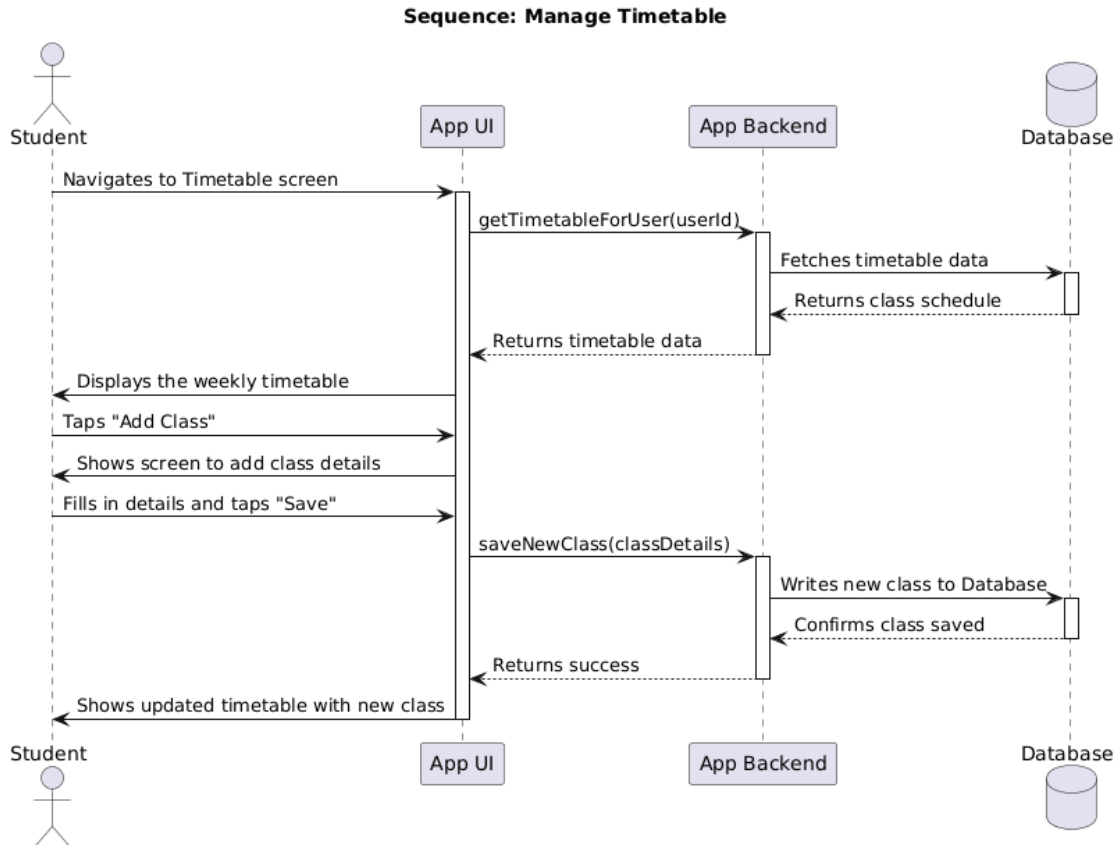


Figure 5. Timetable screen used to manage class scheduling information.

2.2 Gamification and Progression

The second major achievement was the integration of gamification into otherwise ordinary academic actions. Meet & Merit uses XP, levels, credits, and weekly leagues to make productivity more engaging. Completing tasks, finishing focus sessions, and confirming valid social interactions all contribute to progression. This transforms routine actions into a visible feedback loop and gives students a stronger sense of momentum.

Weekly league features extend this system further by allowing students to compare their weekly XP with others. This turns private productivity into a light competitive structure and adds another motivational layer without changing the practical purpose of the app.

During the final phase of development, the progression system was also strengthened through anti-abuse logic. Task XP was limited to the first eight first-time task completions per day, focus XP was limited to 120 rewarded minutes per day, and social XP was limited per student per day while also preventing repeated same-day rewards for the same pair of students. These rules improved the integrity of the reward model without preventing legitimate use of the app after the limit is reached.

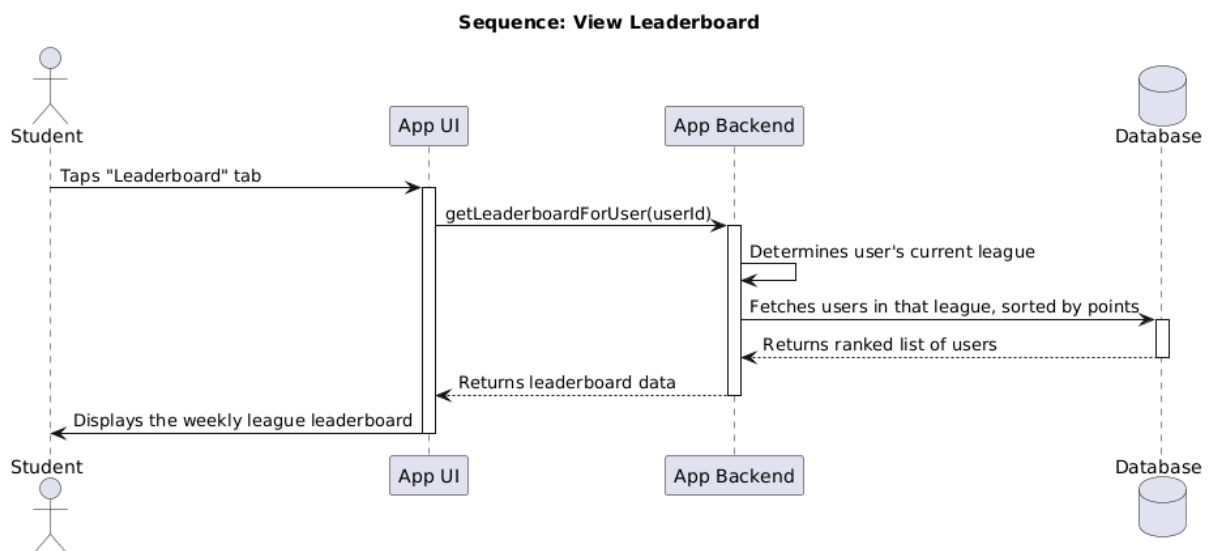


Figure 6. Leaderboard screen showing comparative progression and gamified ranking.

2.3 Nearby Social Features

The most distinctive achievement of the project was the social module. Instead of implementing a conventional chat feature, the app was designed to support nearby student discovery, connection requests, acceptance or rejection, optional friend

creation, and XP rewards for meaningful interaction. This was intentionally aligned with the original purpose of encouraging real-world student engagement rather than screen-based social activity.

The final social flow works through Bluetooth Low Energy discovery supported by backend token resolution. A student enables social mode, a short-lived token is issued by the backend, and nearby devices discover one another through app-specific BLE service data. Once a nearby student is identified, a connection request can be sent, accepted or rejected, and then optionally converted into a friendship. This design is much more defensible than direct generic Bluetooth scanning and is one of the strongest examples of the project improving through iteration.

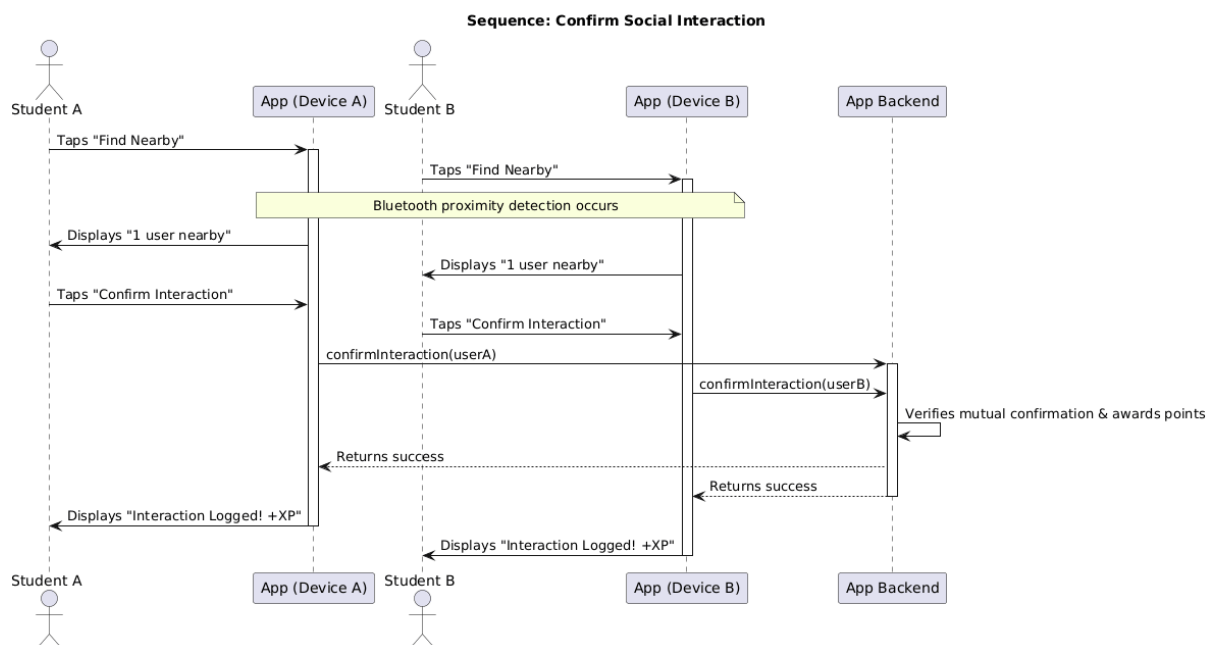


Figure 7. Social interaction confirmation screen used during the nearby connection flow.

2.4 Profile and Avatar Features

The profile system became an important convergence point for progression, identity, and social visibility. The profile page brings together username, XP, level, rank, credits, league information, department, year of study, and avatar. This makes it more than a decorative page; it acts as a summary of the student's progress through the application.

Avatar upload was also completed and later improved through migration to Cloudinary for persistence. This was valuable not only for presentation, but also for usability in nearby discovery, request management, and friend recognition.

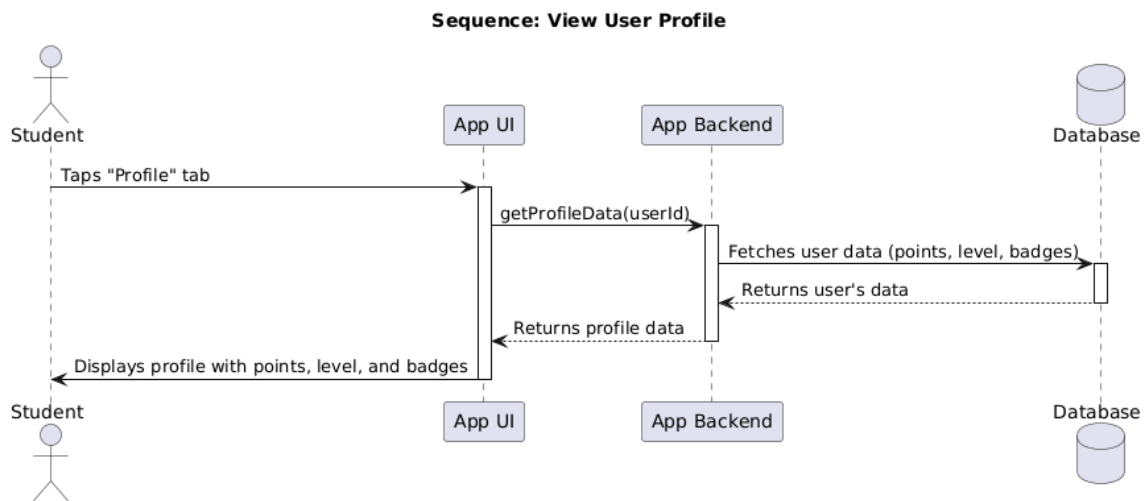


Figure 8. User profile screen showing progression, identity, and social-facing information.

3. Unmet Objectives

Although the project achieved a broad functional scope, several desirable features were not completed within the available time. The most significant unimplemented feature was NFC-based confirmation for social interactions. NFC had been considered as a possible confirmation step for offline encounters, but the Bluetooth redesign already required substantial architectural work, so NFC remained a future enhancement.

Another incomplete area was broader real-world validation of the Bluetooth social workflow. The BLE architecture was significantly improved and tested in a limited multi-device setup, but the project did not include large-scale validation across a wider range of Android hardware, BLE capability differences, or multiple environmental conditions such as crowded spaces and long test sessions.

The application also still relies on a relatively simple `user_id` request pattern rather than stronger token-based authentication and authorisation. This was sufficient for a student project prototype, but it is not a production-ready long-term solution. Related to this, email verification was intentionally not implemented. Although registration collects an email address, full email verification would require additional infrastructure such as token management, verification links, mail delivery configuration, and resend handling. Given the submission timeline, this was treated as a future security-hardening task rather than a core project requirement.

Finally, some richer social extensions were left unfinished, including detailed friendship history, interaction analytics, more advanced moderation or abuse controls, and optional study-session confirmation flows. These would be useful future additions, but they were not essential to demonstrating the core academic and technical aims of the project.

4. Challenges and Resolutions

The most important technical problems encountered during the project were not superficial implementation bugs, but architectural issues that required redesign. These challenges shaped the final form of the application and were a major part of the learning value of the project.

4.1 Bluetooth Identity and Nearby Discovery

The most difficult challenge was the social feature. A naive Bluetooth approach can detect nearby hardware, but it cannot safely and accurately identify which nearby devices correspond to actual users of the application. In realistic environments, generic scanning produces headphones, laptops, printers, smartwatches, and many devices labelled only as unknown hardware. This made the initial discovery approach unsuitable both technically and from a user-experience perspective.

The resolution was to redesign the feature around BLE advertising, app-specific service data, and short-lived backend-issued tokens. Instead of trying to infer user identity from Bluetooth hardware names, the app now advertises a temporary token that must be resolved through the backend into a limited nearby profile. This made the design far more realistic and also improved privacy, because the system no longer depends on broadcasting stable personal identity directly over Bluetooth.

4.2 Deployment and Media Persistence

Another major challenge was the difference between local development and deployed operation. The backend was hosted on Heroku, which required careful management of settings, environment variables, PostgreSQL migrations, and the repository structure used for subtree deployment. Features that behaved correctly in local development still needed to survive deployment, dyno restarts, and migration workflows in production.

This challenge was especially visible in the avatar system. Local file storage worked during early development, but Heroku's file system is ephemeral and therefore unsuitable for persistent user-uploaded media. The solution was to migrate avatar storage to Cloudinary and store only persistent image URLs in the database. This turned a fragile local feature into a more realistic cloud-backed system.

4.3 Reward Integrity and Anti-Abuse Design

As the gamification layer became more complete, a new challenge emerged: the risk of point inflation through repetitive low-value actions. Without additional controls, a user could artificially increase XP through excessive small task completions, repeated focus rewards, or repeated social confirmations.

The resolution was to keep the reward decisions on the backend and introduce daily reward caps. Task completion now tracks whether XP has already been awarded for a task, focus rewards are aggregated per student per day, and social rewards are limited both per pair and per student per day. This was a useful example of the project becoming more robust not by adding a new feature, but by tightening the behaviour of an existing one.

4.4 Validation on Mobile Devices

Several features in the project depend on real mobile behaviour and therefore cannot be validated fully in a desktop-only environment. Bluetooth scanning and advertising, Android permission flows, and device-specific BLE support all required real-device testing. Limited hardware availability made this more difficult than ordinary API or UI validation.

The practical response was to combine layered validation methods: backend tests for business logic, local Android builds for compile safety, and targeted real-device testing for interaction flows. This did not eliminate all testing limitations, but it made the validation process stronger than relying on a single approach.

5. Testing and Validation

Testing was carried out across backend logic, Android build validation, deployment checks, and selected real-device scenarios. Because this project combines mobile hardware behaviour, cloud deployment, and progression rules, validation had to cover more than simple unit-level correctness.

5.1 Backend and Functional Validation

The Django backend was supported by an automated test suite covering major API behaviours such as task handling, timetable creation, notes, league progression, social request confirmation, avatar upload behaviour, and the reward-cap rules added during the final phase. At the current project state, the backend test suite passes in full and provides a useful safety net for regression checking.

The Android frontend was also validated through repeated debug builds. This is important in a project with many iterative UI and navigation changes, because it confirms that the current Kotlin and XML code still compiles cleanly as a complete application rather than only as isolated edited files.

The deployed backend on Heroku was also revalidated after major reward-system changes by redeploying the backend subtree and applying the required migrations. This ensured that the production environment remained aligned with the local codebase.

5.2 Device-Level Validation

Real-device validation focused on the interaction flows most relevant to the app's purpose. These included login and registration, task completion and XP updates, focus mode completion, profile and avatar behaviour, and the social request flow across Android devices. Because Bluetooth support can vary by device, it was especially important to confirm that the BLE-based social architecture could be exercised on actual hardware rather than only reasoned about in code.

The BLE feature was partially constrained by device availability and device capability differences, but the project still progressed from a purely theoretical nearby-social design to a tested multi-device implementation path. This distinction is important: the project did not merely describe a BLE workflow, it implemented one and pushed it into real-device validation as far as the available hardware allowed.

5.3 Validation of Reward Integrity

The final anti-abuse rules were also explicitly validated. Task rewards were checked for the two main exploit paths: exceeding the daily task reward cap and toggling the

same task repeatedly to try to claim XP multiple times. Focus rewards were checked for full reward, partial reward when only some daily reward minutes remain, and zero reward after the daily cap is reached. Social rewards were checked both for per-pair same-day duplication and for per-student daily caps.

This validation was important because reward logic is central to the identity of the application. If the reward system can be exploited easily, then the leaderboard, level progression, and social incentives all become less credible. The additional testing and backend enforcement therefore strengthened not only correctness, but also the overall trustworthiness of the app.

6. Reflections and Lessons Learned

The project provided broad practical experience across mobile development, backend engineering, deployment, and iterative feature design. More importantly, it showed that strong results in software engineering often come less from the first implementation of a feature and more from how that feature is revised after weaknesses become visible.

6.1 Generation of Features Is Easier Than Validation of Features

One of the strongest lessons from the project was that building an initial version of a feature is often easier than making it trustworthy. This was true for Bluetooth discovery, avatar persistence, and reward integrity. In each case, the first implementation seemed plausible, but real use exposed structural weaknesses that required a more careful design.

6.2 Platform Reality Matters

The Bluetooth feature in particular demonstrated that an attractive idea is not enough on its own. Platform behaviour, device support, permissions, payload limits, privacy considerations, and validation strategy all affect whether a feature is genuinely viable. This was a valuable lesson in grounding design decisions in platform reality rather than optimism.

6.3 Full-Stack Projects Reward Clear Separation of Responsibility

The project also reinforced the value of separating responsibilities between frontend and backend. The Android client handles interface and interaction flow, while the backend enforces persistent rules, progression logic, social state transitions, and reward integrity. This separation made later revisions much easier, especially when introducing anti-abuse logic that should not depend on trusting the client.

6.4 Scope Control Is a Technical Skill

Meet & Merit grew to include productivity tools, game mechanics, league logic, profile customisation, media handling, and nearby social discovery. That breadth made the project rich, but it also increased implementation and validation pressure. One of the clearest lessons of the project is that scope control is not only a planning concern; it is also a technical skill that affects quality, testing depth, and delivery confidence.

7. Future Recommendations

The next development cycle should focus less on feature breadth and more on hardening, validation, and selective extension. The following improvements would offer the strongest value.

7.1 Stronger Authentication and Account Verification

The most important production-oriented improvement would be to replace the simple `user_id` request pattern with token-based authentication and stronger authorisation checks. Email verification should also be introduced as part of that hardening effort so that account identity is better controlled.

7.2 Expanded Social Confirmation Options

NFC or another lightweight secondary confirmation mechanism could be added to strengthen the meaning of accepted real-world interactions. This should remain optional and should not replace BLE discovery, since BLE is the more natural discovery layer for the current product direction.

7.3 Broader Real-Device Bluetooth Testing

The BLE feature should be tested across a wider set of Android phones and tablets, especially devices with different BLE advertising support, Android versions, and vendor-specific behaviour. This would improve confidence in the feature's reliability and help distinguish architecture issues from device-specific limitations.

7.4 Richer Social History and Analytics

Future versions of the app could expand friendship history, encounter summaries, and light reflection features around offline interaction. This would add value without turning the app into a conventional chat platform.

7.5 Further Reward-System Tuning

The anti-abuse rules introduced in the final phase improved the reward system, but they should be treated as an initial balancing pass rather than a finished economic model. Future versions should use observed behaviour to refine XP values, daily caps, and the relationship between tasks, focus, and social progression.

8. Conclusion

Meet & Merit delivered a working full-stack student application that combines productivity tools, gamified motivation, and nearby social interaction in one system. The project achieved a substantial feature scope: task management, focus mode, timetable support, notes, XP and level progression, weekly leagues, profile and avatar features, BLE-based nearby discovery, connection requests, and friendships.

The most important result of the project was not simply that these features were implemented, but that the system improved through technical iteration. The Bluetooth social feature became significantly stronger after its original assumptions were challenged. The avatar system became more realistic after persistent cloud storage replaced local file handling. The reward model became more credible after anti-abuse rules were added. These changes made the final system more defensible than its earlier versions.

For those reasons, the project should be viewed as successful both as a software engineering outcome and as a learning experience. It produced a deployable backend, a working Android application, and a clear set of lessons about architecture, scope, platform constraints, testing, and reward integrity. It also leaves a realistic foundation for future development rather than ending as a disconnected prototype.

9. Bibliography

Android Developers *Bluetooth Low Energy overview*. Available at:
<https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview>
(Accessed: 21 April 2026).

Android Developers *Bluetooth permissions*. Available at:
<https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>
(Accessed: 21 April 2026).

Cloudinary *Cloudinary documentation*. Available at:
<https://cloudinary.com/documentation> (Accessed: 21 April 2026).

Django REST Framework *Django REST Framework documentation*. Available at:
<https://www.django-rest-framework.org/> (Accessed: 21 April 2026).

Django Software Foundation *Django documentation*. Available at:
<https://docs.djangoproject.com/> (Accessed: 21 April 2026).

Heroku Dev Center *Python on Heroku*. Available at:
<https://devcenter.heroku.com/categories/python-support> (Accessed: 21 April 2026).

10. Appendices

Appendix A: Core Data Structures

Entity	Purpose	Key Fields
Student	Stores user identity, profile, and progression data	username, email, current_xp, level, credits, department, year_of_study, social_discoverable, avatar
Task	Stores academic tasks together with completion and reward-tracking data	title, is_completed, completed_at, xp_awarded_at, due_date, user
Note	Stores notes linked to study content, tasks, or classes	title, content_markdown, note_type, course_name, linked_task, linked_timetable_entry, user
TimetableEntry	Stores recurring class schedule information	course_name, day_of_week, start_time, end_time, classroom, user
League, LeagueMembership, WeeklyLeagueEntry	Store competitive progression and weekly XP standing	tier, promotion_slots, weekly_xp, final_rank, outcome
DailyFocusReward	Stores focus minutes already rewarded on a given day	student, reward_date, rewarded_minutes
SocialToken	Stores short-lived nearby discovery tokens	student, token_hash, expires_at, is_active

ConnectionRequest, Encounter, Friendship	Store social state, confirmed interactions, and friend relationships	status, confirmed, xp_awarded, created_at
--	--	--

Appendix B: Main Backend Endpoints

Endpoint	Method	Purpose
/api/register/	POST	Create a new account
/api/login/	POST	Authenticate an existing user
/api/tasks/	GET, POST	Retrieve or create tasks
/api/tasks/<id>/	PATCH, DELETE	Update or delete a task
/api/notes/	GET, POST	Retrieve or create notes
/api/notes/<id>/	PATCH, DELETE	Update or delete a note
/api/timetable/	GET, POST	Retrieve or create timetable entries
/api/timetable/<id>/	PATCH, DELETE	Update or delete a timetable entry
/api/profile/	GET	Retrieve profile and progression information
/api/profile/avatar/	POST	Upload a profile avatar
/api/focus/	POST	Award XP for a focus session
/api/league/status/	GET	Retrieve current league status
/api/league/leaderboard/	GET	Retrieve league standings

/api/social/presence/start/	POST	Issue a short-lived BLE token
/api/social/presence/stop/	POST	Deactivate social presence
/api/social/resolve/	POST	Resolve a BLE token to a nearby student profile
/api/social/connect/	POST	Send a connection request
/api/social/confirm/	POST	Accept or reject a connection request
/api/social/requests/	GET	Retrieve social request data
/api/social/friends/	GET, POST	Retrieve or create friendships

